

SOFTWARE ARCHITECTURE DOCUMENT

iKnowBall

Sports Analytics & Prediction Platform — Fullstack Portfolio Project

Target Level: Junior-to-Mid Backend Engineer

Tài liệu này được viết theo tư duy của một Software Architect thiết kế sản phẩm thật, có tính đến ràng buộc thực tế của một sinh viên/dev làm portfolio: chi phí thấp, thời gian có hạn, không over-engineering, nhưng vẫn thể hiện đủ chiều sâu kỹ thuật để phỏng vấn được.

1. Project Overview

iKnowBall là nền tảng web cập nhật số liệu thể thao (bóng đá, bóng rổ/NBA) và xây dựng hệ thống dự đoán xác suất trận đấu dựa trên dữ liệu lịch sử và phong độ.

Giá trị cốt lõi

- Người dùng thường: xem lịch thi đấu, kết quả, BXH, thống kê đội/cầu thủ.
- Người dùng nâng cao: xem dự đoán xác suất, phân tích sâu, theo dõi độ chính xác mô hình theo thời gian.
- Đây không phải công cụ cá cược, không cam kết lợi nhuận — là công cụ phân tích dữ liệu thể thao có định lượng xác suất.

Mục tiêu dự án khi làm portfolio

1. Chứng minh năng lực Fullstack + System Design ở mức "junior-to-mid backend engineer".
2. Có một pipeline dữ liệu thật (không phải data giả), có background job, cache, queue.
3. Có một hệ thống Auth/Authorization/Payment hoàn chỉnh — vì đây là thứ nhà tuyển dụng kiểm tra kỹ nhất.
4. Có một mô hình prediction đơn giản nhưng được đánh giá đúng cách (không chỉ khoe "có AI").
5. Deploy thật, có domain, có CI/CD, để demo trực tiếp khi phỏng vấn.

Điều dự án KHÔNG cần làm (tránh over-engineering)

- Không cần microservices thật sự phân tán (dùng modular monolith là đủ).
- Không cần Kafka (queue nhẹ như BullMQ là đủ).
- Không cần realtime websocket phức tạp cho live score ngay từ đầu (poll 15-30s là đủ cho MVP).
- Không cần Kubernetes để deploy 1 app portfolio.
- Không cần Deep Learning cho prediction — dữ liệu ít, sẽ overfit, và không giải thích được kết quả.

2. Functional Requirements

2.1 Người dùng (Guest / User / Premium)

- Xem lịch thi đấu, kết quả, live score (poll).
- Xem chi tiết trận đấu: đội hình, sự kiện, thống kê.
- Xem trang đội bóng/cầu thủ: thông số mùa giải, phong độ 5-10 trận gần nhất.
- Xem bảng xếp hạng theo giải đấu.
- Đăng ký / đăng nhập / quên mật khẩu / xác thực email / Google OAuth.
- Xem dự đoán trận đấu (giới hạn với Free, đầy đủ với Premium).
- Đăng ký gói Premium, thanh toán, xem lịch sử thanh toán.
- Xem lịch sử dự đoán và độ chính xác của hệ thống theo thời gian.

2.2 Hệ thống

- Đồng bộ dữ liệu thể thao định kỳ từ external API (matches, teams, players, stats).
- Sinh dự đoán tự động trước mỗi trận đấu (batch job, không realtime).
- Cập nhật kết quả thực tế sau trận, đối chiếu với dự đoán để tính accuracy.
- Gửi email (xác thực, reset password, thông báo thanh toán).
- Xử lý webhook thanh toán (idempotent).
- Ghi log audit cho các hành động quan trọng (đăng nhập, thanh toán, thay đổi role).

2.3 Admin

- Quản lý user (khóa/mở khóa, đổi role).
- Theo dõi subscription & payment.
- Theo dõi tình trạng đồng bộ dữ liệu thể thao (job thành công/lỗi).
- Theo dõi performance của prediction model (accuracy, log loss theo thời gian).
- Xem system logs / audit logs.

3. Non-functional Requirements

Nhóm	Yêu cầu	Ghi chú thực tế cho portfolio
Performance	API response < 300ms (cache hit), < 1.5s (cache miss)	Đủ dùng, không cần benchmark tải lớn
Availability	Best-effort, không cần 99.9% SLA	Free-tier hosting, chấp nhận cold-start
Scalability	Thiết kế modular để tách service sau này	Không triển khai microservices thật ngay
Security	OWASP Top 10 cơ bản, hash password, JWT an toàn, input validation	Bắt buộc, đây là điểm cộng lớn khi phỏng vấn
Maintainability	Code có layer rõ ràng (controller-service-repository), test cho phần lỗi	Ưu tiên hơn là 100% coverage
Cost	Chạy được trong free-tier/low-cost tier	Railway/Render/Vercel/Supabase
Observability	Có log tập trung, error tracking cơ bản	Sentry free tier là đủ

4. System Architecture

4.1 Lựa chọn kiến trúc: Monolith vs Modular Monolith vs Microservices

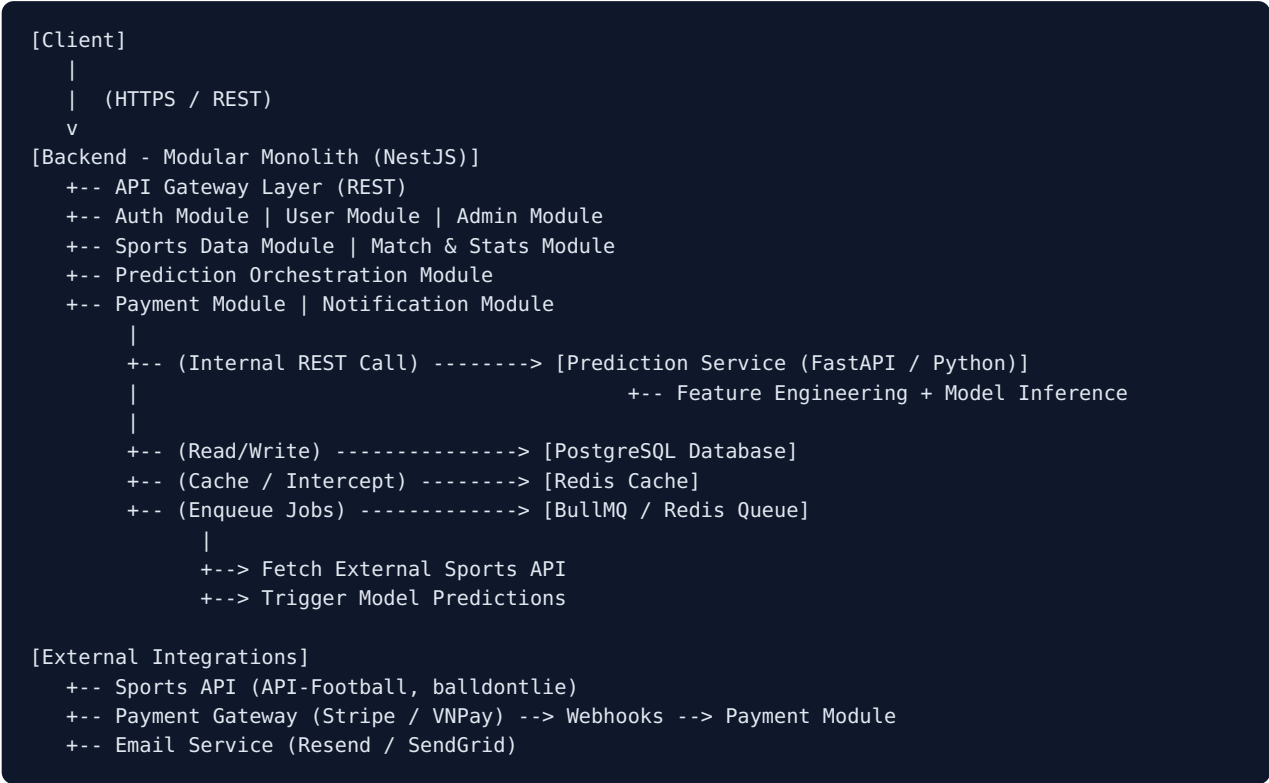
Tiêu chí	Monolith thuần	Modular Monolith (chọn)	Microservices
Độ phức tạp vận hành	Thấp	Thấp-Trung bình	Cao
Chi phí hạ tầng	Thấp	Thấp	Cao (nhiều service, network, service discovery)
Khả năng mở rộng về sau	Kém	Tốt (tách module thành service khi cần)	Tốt nhất nhưng dư thừa cho quy mô nhỏ
Thể hiện System Design khi phỏng vấn	Trung bình	Cao — cho thấy biết thiết kế boundary trước khi phân tán	Cao nhưng dễ bị hỏi ngược "tại sao cần microservices cho app vài nghìn user?"
Thời gian dev cho 1 người	Nhanh	Nhanh-Trung bình	Chậm, tốn effort DevOps

Quyết định: Dùng Modular Monolith cho Backend chính (Auth, User, Sports Data, Match, Prediction-orchestration, Payment, Notification đều là các module tách rõ trong 1 codebase, giao tiếp nội bộ qua service layer, không qua HTTP).

Ngoại lệ duy nhất tách riêng thành service: **Prediction Service** viết bằng Python (do cần scikit-learn/XGBoost), giao tiếp với Backend chính qua REST nội bộ. Đây là ranh giới tách hợp lý vì khác ngôn ngữ/ecosystem, không phải vì lý do scale.

→ Kiến trúc gọi là **"Modular Monolith + 1 Prediction microservice"** — vừa đủ để thể hiện tư duy System Design, vừa không over-engineering.

4.2 High-Level Architecture Diagram



4.3 Frontend Architecture



Tech & lý do chọn

Công nghệ	Lý do
Next.js (App Router) + TypeScript	SSR/SSG giúp SEO cho trang match/team, React Server Components giảm bundle size, TypeScript bắt lỗi sớm và thể hiện tư duy engineering nghiêm túc
Tailwind CSS	Tốc độ dev nhanh, dễ đồng nhất design system, không cần viết CSS riêng cho từng component
TanStack Query (React Query)	Quản lý server state (fetch, cache, refetch cho live score) tốt hơn tự viết useEffect; giảm hẳn boilerplate loading/error state

Công nghệ	Lý do
Zustand	State UI đơn giản (theme, filter, modal) — không cần Redux vì app không có state phức tạp liên module
Recharts	Đủ cho biểu đồ phong độ, radar chart chỉ số cầu thủ, không cần D3 thô
Zod	Validate response/schema phía client, đồng bộ type với backend nếu dùng chung schema

Không dùng: Redux Toolkit (dư thừa), Redux-Saga, GraphQL Apollo (REST đã đủ và dễ debug hơn cho portfolio).

4.4 Backend Architecture (Modular Monolith — NestJS)

Vì sao chọn NestJS thay vì Express thuần

- NestJS có sẵn kiến trúc module hóa (Module - Controller - Service - Repository/Provider), Dependency Injection, Guards (cho AuthZ), Interceptors (cho logging/caching), Pipes (validation) — đúng là những khái niệm System Design mà interviewer sẽ hỏi.
- Express thuần thì linh hoạt nhưng phải tự dựng lại toàn bộ cấu trúc này → tốn effort mà không tăng điểm phỏng vấn tương ứng.
- NestJS dùng TypeScript native, đồng bộ ngôn ngữ với Frontend.

Cấu trúc module nội bộ

```

AppModule
+-- AuthModule (register, login, refresh, oauth, guards)
+-- UserModule (profile, role, subscription status)
+-- SportsDataModule (ingestion, sync jobs, external API adapters)
+-- MatchModule (matches, teams, players, standings, stats)
+-- PredictionModule (gọi Prediction Service, lưu kết quả, đánh giá accuracy)
+-- PaymentModule (subscription, payment gateway integration, webhook)
+-- NotificationModule (email, in-app notification)
+-- AdminModule (quản trị, dashboard nội bộ)
+-- SharedModule (cache service, queue service, logger, config)

```

Giao tiếp giữa các module: gọi trực tiếp qua service interface (dependency injection) trong cùng process — không qua HTTP nội bộ, trừ Prediction Service (Python) là REST call thật vì khác runtime.

REST vs GraphQL: Chọn REST. Lý do: dữ liệu thể thao có cấu trúc phân cấp rõ ràng (match → teams → players → stats), REST dễ cache theo URL (quan trọng vì cache là điểm cốt lõi của hệ thống), dễ document bằng OpenAPI/Swagger, và dễ giải thích trong phỏng vấn hơn GraphQL resolver phức tạp. GraphQL sẽ là "nice to have" ở phần Future Improvements, không phải nhu cầu thật ở quy mô này.

Các cross-cutting concerns

Concern	Giải pháp
Authentication	JWT (access + refresh token), Passport.js strategies (local, jwt, google)
Authorization	NestJS Guards + Roles decorator (RBAC)
Validation	class-validator + DTO ở mọi endpoint
Rate limiting	@nestjs/throttler (per-IP, per-user cho endpoint nhạy cảm: login, payment)
Caching	Redis, cache-aside pattern qua Interceptor tùy chỉnh
Logging	Pino logger (structured JSON log) + request-id tracing
Background jobs	BullMQ (Redis-based queue)
Config/secrets	@nestjs/config + .env, secrets thật để trên hosting platform (Railway/Render secret manager), không commit

5. Database Design (PostgreSQL)

Vì sao PostgreSQL, không phải MongoDB: dữ liệu có quan hệ chặt (match ↔ teams ↔ players ↔ stats ↔ prediction), cần transaction cho payment, cần JOIN cho các truy vấn thống kê (head-to-head, form). PostgreSQL còn hỗ trợ JSONB cho các trường dữ liệu thô/linh hoạt từ external API (ví dụ raw stats payload) — kết hợp tốt cả 2 nhu cầu.

5.1 Danh sách Entity

User & Access

- **User:** id (PK, uuid), email (unique), password_hash, full_name, avatar_url, email_verified_at, role_id (FK), status (active/banned), created_at, updated_at. Index: email.
- **Role:** id (PK), name (guest/user/premium/admin), permissions (jsonb).
- **RefreshToken:** id (PK), user_id (FK), token_hash, expires_at, revoked_at, device_info. Index: user_id, token_hash.
- **OAuthAccount:** id (PK), user_id (FK), provider (google), provider_user_id, created_at. Unique: (provider, provider_user_id).
- **PasswordResetToken:** id (PK), user_id (FK), token_hash, expires_at, used_at.

Subscription & Payment

- **Subscription:** id (PK), user_id (FK), plan (free/premium), status (active/expired/canceled), current_period_end, created_at. Index: user_id, status.
- **Payment:** id (PK), user_id (FK), subscription_id (FK), gateway (stripe/vnpay/momo), gateway_transaction_id (unique), amount, currency, status (pending/success/failed/refunded), raw_payload (jsonb), created_at. Index: gateway_transaction_id, user_id.
- **WebhookEvent** (idempotency log): id (PK), gateway, event_id (unique), payload (jsonb), processed_at.

Sports Domain

- **Sport:** id (PK), name (football/basketball).
- **League:** id (PK), sport_id (FK), name, country, external_id (unique, từ sports API), season.
- **Team:** id (PK), league_id (FK), name, short_name, logo_url, external_id (unique), founded_year.
- **Player:** id (PK), team_id (FK), full_name, position, nationality, date_of_birth, external_id (unique).

- **Match:** id (PK), league_id (FK), home_team_id (FK), away_team_id (FK), match_date, status (scheduled/live/finished/postponed), home_score, away_score, external_id (unique), raw_data (jsonb). Index: match_date, league_id, (home_team_id, away_team_id).
- **MatchEvent:** id (PK), match_id (FK), type (goal/card/substitution), minute, player_id (FK), team_id (FK), detail (jsonb). Index: match_id.
- **TeamStats:** id (PK), team_id (FK), league_id (FK), season, matches_played, wins, draws, losses, goals_for, goals_against, elo_rating, updated_at. Unique: (team_id, league_id, season).
- **PlayerStats:** id (PK), player_id (FK), match_id (FK, nullable cho season-aggregate), season, minutes_played, goals, assists, rating, advanced_metrics (jsonb — xG, PER, v.v.). Index: player_id, match_id.
- **Standing:** id (PK), league_id (FK), team_id (FK), season, rank, points, played, won, drawn, lost. Unique: (league_id, team_id, season).

Prediction

- **Prediction:** id (PK), match_id (FK), model_version, home_win_prob, draw_prob (nullable cho basketball), away_win_prob, predicted_outcome, features_snapshot (jsonb), created_at. Unique: (match_id, model_version). Index: match_id.
- **PredictionResult:** id (PK), prediction_id (FK), actual_outcome, is_correct (bool), log_loss, brier_score, evaluated_at. Index: prediction_id.
- **ModelPerformance** (aggregate theo thời gian để vẽ dashboard): id (PK), model_version, league_id (FK, nullable), period_start, period_end, accuracy, precision, recall, f1, avg_log_loss, avg_brier_score, sample_size.

System

- **Notification:** id (PK), user_id (FK), type, title, message, is_read, created_at. Index: user_id.
- **AuditLog:** id (PK), actor_user_id (FK, nullable), action, target_type, target_id, metadata (jsonb), ip_address, created_at. Index: actor_user_id, created_at.
- **SyncJobLog** (theo dõi tình trạng ingestion): id (PK), job_name, status (success/failed/partial), started_at, finished_at, records_processed, error_message.

5.2 ERD Outline

```
[ROLE] 1--N [USER] 1--N [REFRESH_TOKEN]
      |
      | 1--N [OAUTH_ACCOUNT]
      | 1--N [SUBSCRIPTION] 1--N [PAYMENT]
      | 1--N [NOTIFICATION]
      |
[SPORT] 1--N [LEAGUE] 1--N [TEAM] 1--N [PLAYER]
      |           |           |
      |           |           |
      +-- N [MATCH] ---+-----+----- N [PLAYER_STATS]
      |           |           |
      |           |           |
      |           +-- N [MATCH_EVENT]
      |
      +-- N [TEAM_STATS]
      |
[MATCH] 1--1 [PREDICTION] 1--1 [PREDICTION_RESULT]
      |
      +-- N [STANDING]
```

6. API Design (REST)

Quy ước: mọi response bọc trong `{ data, meta, error }`; phân trang qua `?page&limit`; versioning qua prefix `/api/v1`.

6.1 Auth

Endpoint	Method	Auth	Mô tả
<code>/api/v1/auth/register</code>	POST	Không	Đăng ký, gửi email verify
<code>/api/v1/auth/login</code>	POST	Không	Trả access token + set refresh token (httpOnly cookie)
<code>/api/v1/auth/refresh</code>	POST	Refresh cookie	Cấp access token mới
<code>/api/v1/auth/logout</code>	POST	Access token	Thu hồi refresh token
<code>/api/v1/auth/verify-email</code>	POST	Không	Xác thực email (token trong body)
<code>/api/v1/auth/forgot-password</code>	POST	Không	Gửi email reset
<code>/api/v1/auth/reset-password</code>	POST	Không	Đặt lại mật khẩu (token trong body)
<code>/api/v1/auth/google</code>	GET	Không	Redirect OAuth Google
<code>/api/v1/auth/google/callback</code>	GET	Không	Callback, tạo/liên kết user

6.2 User

Endpoint	Method	Auth	Authorization	Mô tả
<code>/api/v1/users/me</code>	GET	Bắt buộc	Chính chủ	Lấy profile + subscription status
<code>/api/v1/users/me</code>	PATCH	Bắt buộc	Chính chủ	Cập nhật profile

6.3 Sports Data (public, cache mạnh)

Endpoint	Method	Auth	Mô tả
<code>/api/v1/leagues</code>	GET	Không	Danh sách giải đấu
<code>/api/v1/leagues/:id/standings</code>	GET	Không	BXH

Endpoint	Method	Auth	Mô tả
/api/v1/matches? date=&league=&status=	GET	Không	Danh sách trận (lịch/kết quả/live)
/api/v1/matches/:id	GET	Không	Chi tiết trận + events + stats
/api/v1/matches/:id/h2h	GET	Không	Lịch sử đối đầu 2 đội
/api/v1/teams/:id	GET	Không	Thông tin đội + form gần đây
/api/v1/teams/:id/stats?season=	GET	Không	Thống kê mùa giải
/api/v1/players/:id	GET	Không	Thông tin + thống kê cầu thủ

6.4 Prediction

Endpoint	Method	Auth	Authorization	Mô tả
/api/v1/predictions? league=&date=	GET	Không	Free: giới hạn/ngày	Danh sách dự đoán
/api/v1/ predictions/:matchId	GET	Tùy	Premium full, Free tóm tắt	Chi tiết xác suất & features
/api/v1/predictions/ performance	GET	Không	—	Dashboard accuracy public
/api/v1/predictions/ performance/history	GET	Bắt buộc	Premium	Chi tiết theo league/thời gian

6.5 Subscription & Payment

Endpoint	Method	Auth	Mô tả
/api/v1/subscriptions/plans	GET	Không	Danh sách gói
/api/v1/subscriptions	POST	Bắt buộc	Tạo yêu cầu subscribe → trả payment URL
/api/v1/payments/:id	GET	Bắt buộc, chính chủ	Trạng thái giao dịch
/api/v1/payments/ webhook/:gateway	POST	Signature verify	Nhận callback từ gateway (không JWT)

6.6 Admin (role = admin)

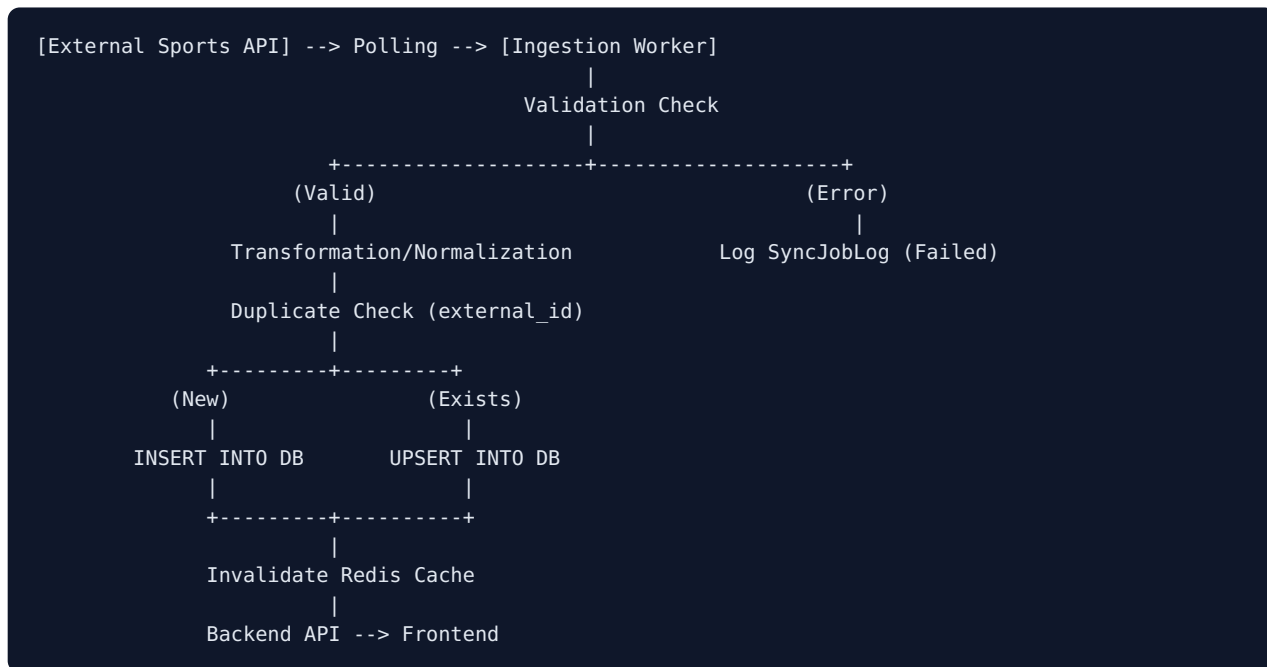
Endpoint	Method	Mô tả
/api/v1/admin/users	GET / PATCH	Danh sách, ban/unban, đổi role
/api/v1/admin/payments	GET	Theo dõi giao dịch
/api/v1/admin/sync-jobs	GET	Trạng thái job đồng bộ dữ liệu
/api/v1/admin/model-performance	GET	Chi tiết accuracy model
/api/v1/admin/audit-logs	GET	Nhật ký hành động

Role	Quyền hạn
Premium User	Toàn bộ prediction chi tiết (feature breakdown, xác suất đầy đủ), advanced stats, lịch sử dự đoán cá nhân, export dữ liệu cơ bản
Admin	Toàn quyền quản trị hệ thống, không truy cập trực tiếp dữ liệu thanh toán thẻ (chỉ xem trạng thái giao dịch qua gateway)

Triển khai bằng NestJS Guards: `@UseGuards(JwtAuthGuard, RolesGuard) + @Roles('premium', 'admin')` decorator, kiểm tra thêm subscription còn hạn hay không (không chỉ role) vì Premium hết hạn phải tự động fallback về Free.

8. Sports Data Architecture

8.1 Pipeline tổng thể



8.2 Polling vs Webhook

Hầu hết Sports API free/rẻ (API-Football, TheSportsDB, balldontlie) không hỗ trợ webhook → chiến lược chính là polling có lịch trình khác nhau theo trạng thái trận đấu:

Trạng thái	Tần suất polling
Trận sắp diễn ra (>24h)	1 lần / 6 giờ (cập nhật lịch, đổi lịch)
Trận trong ngày, chưa bắt đầu	1 lần / 30 phút
Trận đang diễn ra (live)	1 lần / 15-30 giây (chỉ với trận đang live, không poll toàn bộ)
Trận đã kết thúc	1 lần ngay sau khi kết thúc để chốt kết quả cuối, sau đó dừng
Thống kê mùa giải / BXH	1 lần / ngày

→ Đây chính là lý do "live score" khả thi ở mức near-real-time (15-30s) mà không cần WebSocket/SSE ở MVP — đơn giản hóa đáng kể so với việc dựng hạ tầng realtime thật.

8.3 Xử lý lỗi & độ tin cậy

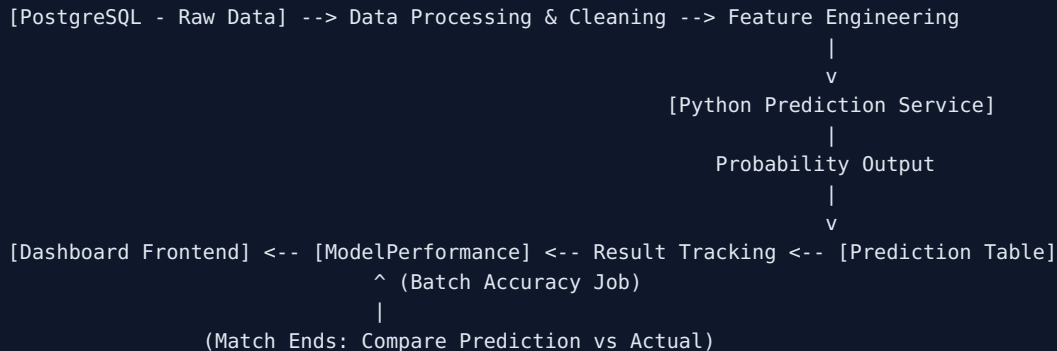
- **Rate limit:** áp dụng token-bucket ở tầng adapter gọi external API; nếu 429 → backoff theo cấp số nhân (exponential backoff) và requeue job.
- **Timeout:** timeout 5-10s mỗi call, retry tối đa 3 lần với backoff, sau đó ghi SyncJobLog(status=failed) và không làm sập toàn bộ batch job (isolate theo từng match/team).
- **Duplicate data:** mọi entity từ external API lưu kèm external_id (unique constraint) → dùng UPSERT (ON CONFLICT DO UPDATE) thay vì insert thô.
- **Đồng bộ dữ liệu:** dùng updated_at/hash checksum của payload để chỉ ghi lại khi có thay đổi thật, tránh ghi log/thay đổi cache không cần thiết.
- **Data normalization:** chuẩn hóa tên đội/cầu thủ, timezone (lưu UTC, convert ở FE), đơn vị số liệu (vd đổi field xg/expected_goals về cùng tên chuẩn nội bộ) qua một lớp Adapter/Mapper riêng cho từng nguồn API — để khi đổi API provider chỉ cần viết Adapter mới, không đổi domain model.

8.4 Đề xuất Sports API

API	Ưu điểm	Phù hợp
API-Football (API-Sports)	Dữ liệu bóng đá rất đầy đủ (live, stats, odds), free tier 100 req/ngày	Nguồn chính cho Football
balldontlie.io	Miễn phí hoàn toàn, dữ liệu NBA (teams, players, stats, games)	Nguồn chính cho Basketball
TheSportsDB	Free, có logo/metadata đội, backup khi API chính rate-limit	Metadata bổ sung

9. Prediction Architecture

9.1 Pipeline



9.2 Feature đề xuất

Chung (cả 2 môn)

- Recent form (kết quả 5-10 trận gần nhất, quy đổi điểm số)
- Home/Away performance riêng biệt (nhiều đội mạnh sân nhà, yếu sân khách)
- Head-to-head (tỷ lệ thắng/thua trong N lần gặp gần nhất)
- Elo rating (tự tính, cập nhật sau mỗi trận — là feature định lượng sức mạnh tốt và dễ giải thích)
- Rest days (số ngày nghỉ trước trận, ảnh hưởng thể lực)
- Strength of opponent (Elo đối thủ)

Bóng đá: Average goals scored/conceded, Expected Goals (xG), Tỷ lệ giữ sạch lưới (clean sheet rate).

Bóng rổ (NBA): Offensive/Defensive Rating, Pace, Điểm trung bình ghi/thủng, Point differential, Ảnh hưởng cầu thủ chủ chốt vắng mặt.

9.3 Roadmap model theo giai đoạn

Phase	Approach	Model	Lý do
Phase 1 (MVP)	Rule-based / thống kê	Elo rating + tỷ lệ thắng theo form, quy đổi qua hàm logistic đơn giản	Không cần data training lớn, giải thích 100%, đủ demo pipeline & đo accuracy
Phase 2	Machine Learning cổ điển	Logistic Regression → XGBoost/LightGBM	Gradient Boosting vượt trội Neural Network ở dữ liệu dạng bảng (tabular)

Phase	Approach	Model	Lý do
Phase 3 (optional)	ML nâng cao	Ensemble (Logistic + XGBoost) + Platt scaling calibration	Chỉ làm khi Phase 2 đã ổn định; không dùng Deep Learning để tránh overfit

10. Prediction Accuracy & Evaluation

10.1 Vì sao không chỉ dùng Accuracy

Accuracy chỉ đo tỷ lệ đoán đúng outcome (thắng/thua/hòa) nhưng không đo chất lượng của xác suất đưa ra. Một model đoán "60%-40%" và một model đoán "99%-1%" cho cùng 1 trận thắng đều được tính "đúng" như nhau nếu chỉ nhìn accuracy — nhưng model thứ hai quá tự tin một cách nguy hiểm (overconfident) nếu sai.

Metric	Đo cái gì	Khi nào quan trọng
Accuracy	Tỷ lệ đoán đúng outcome	Dễ hiểu cho người dùng cuối, nhưng dễ gây hiểu lầm
Precision / Recall / F1	Chất lượng theo từng lớp outcome	Phát hiện model bias về 1 outcome phổ biến
Log Loss	Phạt nặng khi model tự tin sai	Đánh giá chất lượng xác suất — quan trọng nhất với bài toán này
Brier Score	Sai số bình phương giữa xác suất dự đoán & outcome thực tế	Trực quan hơn Log Loss, dễ trình bày trên dashboard
Accuracy theo league / time	Phát hiện model bị suy giảm theo thời gian (data drift)	Ra quyết định retrain

10.2 Dashboard Model Performance (public, minh bạch)

- Biểu đồ accuracy theo tuần/tháng (line chart).
- Bảng so sánh: Model vs Baseline ngẫu nhiên vs Baseline "luôn chọn đội mạnh hơn".
- Phân rã theo giải đấu (Premier League, La Liga, NBA...).
- Log loss & Brier score trend.

Disclaimer cố định: "Đây là công cụ phân tích xác suất dựa trên dữ liệu lịch sử, không phải lời khuyên cá cược và không đảm bảo kết quả."

11. Payment & Subscription Architecture

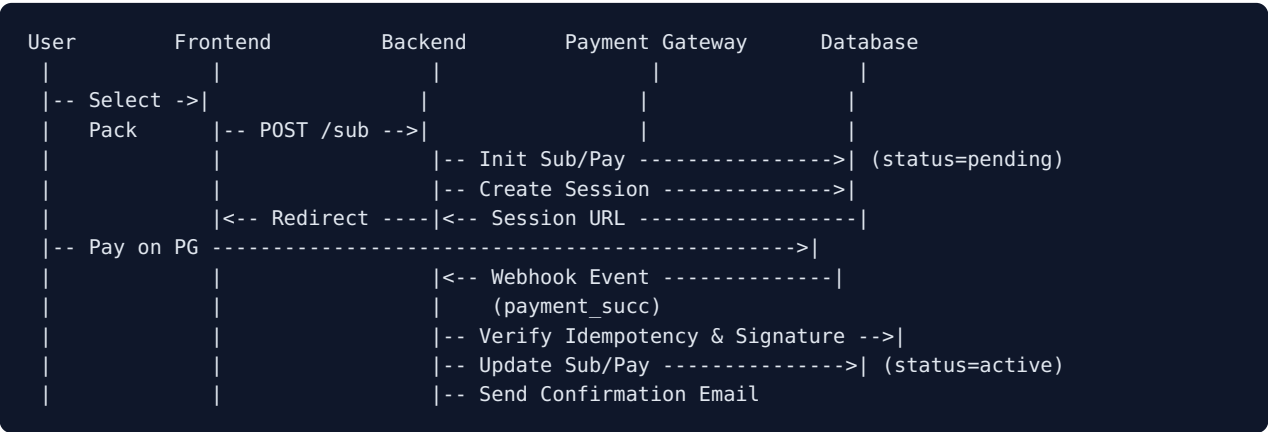
11.1 Phân quyền theo gói

Tính năng	Free	Premium
Lịch thi đấu, kết quả, BXH	✓	✓
Thống kê cơ bản đội/cầu thủ	✓	✓
Prediction	Giới hạn (3 trận/ngày, chỉ xem outcome)	Full xác suất + feature breakdown
Lịch sử dự đoán cá nhân	✗	✓
Advanced stats (Elo, xG, pace...)	✗	✓
Biểu đồ nâng cao (radar, trend)	✗	✓
API access (API key riêng)	✗	✓ (rate-limited)

11.2 Payment Gateway đề xuất

Stripe (test mode) — SDK tốt, sandbox test dễ, tài liệu chuẩn, là gateway phổ biến nhất trong portfolio dev toàn cầu.

11.3 Flow



11.4 Xử lý các trường hợp

- **Payment success:** verify chữ ký webhook (HMAC secret của gateway) trước khi xử lý, activate subscription, ghi Payment.status=success.
- **Payment failed:** giữ Subscription.status=pending, thông báo user, cho phép retry.

- **Refund:** endpoint admin trigger refund qua gateway API, cập nhật `Payment.status=refunded`, hạ cấp subscription về Free.
- **Subscription expired:** background job (cron hằng ngày) quét `Subscription.current_period_end < now()` → chuyển `status=expired`, hạ role về Free.
- **Idempotency:** mọi webhook lưu `event_id` unique trước khi xử lý logic nghiệp vụ.
- **Transaction logging:** mọi thay đổi trạng thái `Payment/Subscription` ghi vào `AuditLog` kèm `actor = "system:webhook"`.

12. Caching (Redis)

Dữ liệu	Chiến lược	TTL đề xuất
Live score đang diễn ra	Cache-aside, invalidate khi worker update	15-30s
Danh sách trận theo ngày	Cache-aside	5 phút (10-15s nếu có live)
Chi tiết team/player	Cache-aside	1-6 giờ
BXH (Standing)	Cache-aside	30-60 phút
Prediction đã sinh	Cache-aside, key theo match_id + version	Đến khi trận kết thúc
Model performance dashboard	Cache-aside	15-30 phút

13. Background Jobs & Queue

Job	Trigger	Mô tả
sync-matches	Cron	Gọi Sports API, upsert Match/Team/Player/Stats
sync-live-scores	Cron mỗi 15-30s (khi live)	Cập nhật score/events, invalidate cache
generate-predictions	Cron hằng ngày (6h sáng)	Gọi Prediction Service, lưu Prediction
evaluate-predictions	Cron sau trận (~2-3h)	So khớp result, ghi PredictionResult & ModelPerformance
process-payment-webhook	Event-driven	Xử lý nặng (DB, email), trả 200 nhanh cho gateway
send-notification	Event-driven	Gửi email xác thực, reset pass, payment receipt
expire-subscriptions	Cron hằng ngày	Hạ cấp subscription hết hạn

Công nghệ: **BullMQ** (trên Redis) — đủ mạnh cho toàn bộ nhu cầu trên (delay job, retry, cron-like repeatable job, concurrency control).

14. Security

Vấn đề	Triển khai	Mức độ ưu tiên
Password hashing	bcrypt cost 12	Bắt buộc
JWT security	access token ngắn hạn, refresh token httpOnly cookie	Bắt buộc
Input validation	class-validator DTO ở mọi endpoint	Bắt buộc
SQL Injection	dùng ORM (Prisma/TypeORM) với parameterized query	Bắt buộc
XSS	Next.js tự escape mặc định, sanitize input, CSP header	Bắt buộc
CSRF	Bearer token + SameSite=Strict cookie	Bắt buộc
Rate limiting	Throttler cho auth & payment endpoints	Bắt buộc
Secure headers	Helmet.js (HSTS, X-Frame-Options)	Bắt buộc
CORS	Whitelist domain frontend cụ thể	Bắt buộc
Payment webhook verify	Verify chữ ký HMAC từ gateway	Bắt buộc
Audit logs	Ghi log hành động nhạy cảm (login, role, payment)	Nên có

15. Testing Strategy

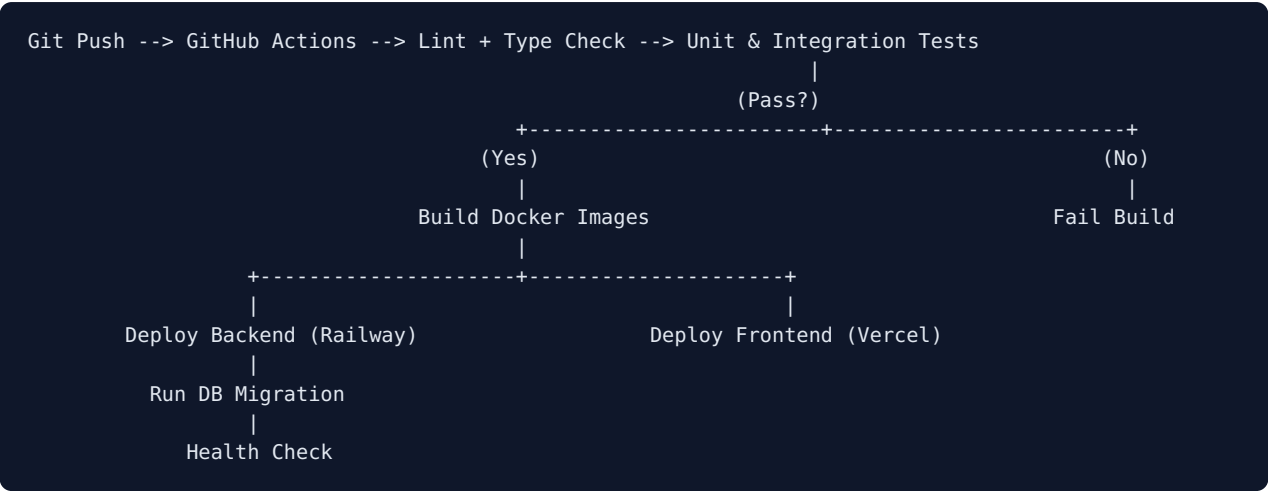
Tầng	Loại test	Công cụ	Phạm vi ưu tiên
Frontend	Unit test	Vitest	Hàm format/tính toán %
Frontend	Component test	React Testing Library	MatchCard, PredictionCard, LoginForm
Frontend	E2E	Playwright	Register → Login → View match → Subscribe
Backend	Unit test	Jest	Elo calc, accuracy calc, RBAC guard
Backend	Integration test	Jest + Testcontainers	Auth flow, Payment idempotency, Sync upsert
Backend	API/contract test	Supertest	Mọi public endpoint quan trọng
Prediction	Backtesting	Python scripts	Mô phỏng accuracy lịch sử theo thời gian
Prediction	Data leakage check	Assertions tự động	Đảm bảo không dùng feature tương lai

16. DevOps & Deployment

16.1 Nền tảng hosting (chi phí tối ưu)

Thành phần	Đề xuất	Lý do
Frontend (Next.js)	Vercel (free)	Tối ưu Next.js, CI/CD tự động, preview deployments
Backend (NestJS)	Railway / Render	Deploy Docker dễ, free Postgres/Redis đi kèm
Prediction Service	Railway / Render	Container riêng, private network communication
Database	Supabase / Railway	Supabase có UI management tốt để demo
Redis	Upstash / Railway	Serverless Redis phù hợp cho project nhỏ

16.2 CI/CD Workflow



16.3 Observability

Application Logs: Pino (JSON) | Error Tracking: Sentry (Free) | Uptime: UptimeRobot | Queue UI: Bull Board.

17. MVP Scope

MVP PHẢI CÓ:

1. Auth đầy đủ + RBAC (Free / Premium / Admin) — 1 môn thể thao (Bóng đá).
2. Sports Data pipeline thật: sync job từ API-Football, lưu Postgres, cache Redis, hiển thị match/stats.
3. Prediction Phase 1 (Elo + Logistic) cho trận 48h tới, lưu Prediction & PredictionResult.
4. Dashboard accuracy công khai + Payment Stripe test-mode + Admin dashboard cơ bản.
5. Deploy production (Vercel + Railway/Render + Supabase) + CI/CD.

18. Development Roadmap

Phase	Mục tiêu	Tasks chính	Output
0 — Planning	Chốt scope, kiến trúc	Viết design doc, setup repo, chọn API	Design doc + Repo skeleton
1 — Setup	Khung dự án chạy được	Setup NestJS, Next.js, Docker compose, Prisma	docker-compose up OK
2 — Auth	Auth/RBAC hoàn chỉnh	Register/Login/Refresh, Google OAuth, Guards	Auth module test pass
3 — Sports Data	Pipeline dữ liệu thật	Adapter API-Football, BullMQ worker, upsert logic	Dữ liệu thật trong DB
4 — Match & Stats	Hiển thị FE	API + FE pages: schedule, standings, match detail	Pages active, Cache hit <1s
5 — Prediction	Dự đoán & Đánh giá	Elo calculator, FastAPI prediction service, evaluate job	FE Prediction + Dashboard
6 — Payment	Monetization flow	Stripe checkout, webhook idempotent, subscription gate	Role upgrade via payment
7 — Admin	Quản trị hệ thống	User management, sync job monitor, accuracy monitor	Admin Dashboard UI
8 — Testing	Đảm bảo chất lượng	Unit/integration test backend, E2E test, backtesting	CI Test suite passing
9 — Deploy	Production release	Dockerize, GitHub Actions CI/CD, deploy Vercel/Railway	Public Live Demo Link
10 — Expand	Tinh chỉnh & Mở rộng	Thêm NBA Basketball, XGBoost model	Version 2.0 Release

19. Folder Structure

```
iKnowBall/
+-- frontend/                                # Next.js App Router
|   +-- app/
|   |   +-- (public)/matches, teams, players, leagues, predictions/
|   |   +-- (auth)/login, register, verify-email, reset-password/
|   |   +-- (protected)/profile, subscription/
|   |   +-- (admin)/dashboard/
|   +-- components/
|   +-- lib/                                # api client, zod schemas
|   +-- hooks/
|   +-- store/                              # zustand
|
+-- backend/                                # NestJS Modular Monolith
|   +-- src/
|   |   +-- auth/
|   |   +-- user/
|   |   +-- sports-data/                    # adapters/, jobs/
|   |   +-- match/
|   |   +-- prediction/                     # HTTP client call to Python service
|   |   +-- payment/                       # gateways/stripe.gateway.ts
|   |   +-- notification/
|   |   +-- admin/
|   |   +-- shared/                         # cache, queue, logger, guards
|   +-- prisma/schema.prisma
|   +-- test/
|
+-- prediction-service/                     # Python FastAPI
|   +-- app/
|   |   +-- features/                       # feature engineering
|   |   +-- models/                         # elo.py, logistic_model.py, xgboost_model.py
|   |   +-- evaluation/                     # metrics.py (log loss, brier score)
|   +-- notebooks/                          # backtesting, EDA
|
+-- infrastructure/                         # Dockerfiles, docker-compose.yml
+-- README.md
```

20. Portfolio & Interview Strategy

Điểm nhấn kỹ thuật cho CV:

- **Architecture:** Modular Monolith kết hợp Python Microservice, giải thích rõ trade-offs.
- **Data Pipeline:** Data ingestion-validation-normalization với idempotent upsert & Redis caching.
- **Security & Payment:** Webhook idempotency, JWT refresh token rotation, RBAC guards.
- **ML Evaluation:** Đánh giá định lượng xác suất qua Log Loss / Brier Score, chống data leakage trong backtesting.

21. Key Interview Questions Prepared

- **System Design:** Vì sao chọn Modular Monolith thay vì Microservices? Nếu traffic x100 thì scale phần nào trước?
- **Database:** Tại sao dùng external_id thay vì PK của API bên ngoài? Chiến lược indexing cho bảng Match?
- **Prediction/ML:** Làm sao đảm bảo không bị Data Leakage? Vì sao chọn Log Loss thay vì chỉ dùng Accuracy?
- **Security:** Idempotency trong Webhook xử lý cụ thể thế nào? Chống Race Condition ra sao?

22. Future Improvements

- Mở rộng thêm môn thể thao (Tennis, Volleyball).
- Nâng cấp Prediction lên ensemble model + calibration.
- WebSocket/SSE cho live score realtime khi scale lớn.
- Đa gateway thanh toán (VNPay/MoMo).
- Public API Key billing cho 3rd party developers.